

Technical Implementation Plan

E-Commerce Platform Development Plan for Developer

Document Version: 1.0	Target Audience: Lead Developer / Dev Team
Project: Modern Apparel E-Commerce Store	Project Scope: Customer Portal & Admin Panel

1 Technical Architecture & Recommended Stack

This plan outlines the end-to-end technical construction of a high-performing apparel store platform. It provides clear architectural guidelines, database schemas, sprint breakdowns, and quality assurance criteria for the development team.

1.1 Technology Stack Recommendations

Layer		Recommended Tech	Implementation Notes
Frontend	Framework	Next.js 14+ (App Router) or React + Vite	SSR / SSG for lightning fast SEO performance on catalog and product pages.
	Styling & UI Kit	Tailwind CSS + Radix UI / Shadcn UI	Accessible pre-built UI primitives (Drawers, Modals, Accordions, Tooltips).
	Micro-Animations	Framer Motion + Lucide Icons	Smooth micro-interactions for slide-out cart, dynamic badges, and visual drawers.
	State Management	Zustand or React Context API	Global state management for shopping cart, active currency, wishlist, and search.
Data Visualization		Recharts	Interactive charts for the Admin Analytics Dashboard (Revenue, AOV, Conversion).
Backend / Database		Node.js + PostgreSQL (Prisma ORM)	Relational database required for matrix options (Variants × Sizes × Colors).
Payments & Check-out		Stripe API + Apple/Google Pay	PCI-compliant hosted fields, BNPL (Klarna / Afterpay) integration support.
Email Notifications		Gmail API / SendGrid / Nodemailer	Automated order receipts, dispatch confirmations, and password reset workflows.
Location & Maps		Google Maps Platform API	Address auto-complete at checkout and interactive carrier/order tracking route map.
SMS Gateway		Twilio / SMS API	Real-time SMS notifications for order updates, OTP verification, and delivery alerts.

Table 1: System Technology Stack

2 12-Week Development Sprint Roadmap

2.1 Phase 1: Architecture, Design System & State Setup (Weeks 1–2)

- Configure monorepo/repository structure with TypeScript, ESLint, and Prettier rules.
- Set up Tailwind CSS design tokens: primary slate (#0F172A), soft background (#F8FAFC), accent rose (#E11D48), and clean typography.
- Implement foundational UI primitives using Shadcn/Radix (Buttons, Inputs, Modals, Drawers).
- Configure Zustand global store modules:
 - useCartStore: Cart items, quantities, promo codes, and dynamic shipping threshold.
 - useWishlistStore: Saved items, back-in-stock alerts.
 - useCurrencyStore: Currency symbol and conversion multipliers (\$, , £).
 - useFilterStore: Active filtering and sorting criteria.
- Seed initial mock database with 8 full apparel products with matrix variants, reviews, and order histories.

2.2 Phase 2: Navigation, Search & Product Listing Page (Weeks 3–4)

- **Sticky Navigation Header:** Multi-level category menu (Men, Women, Accessories, Sale), dynamic search trigger, currency switcher, wishlist badge, and cart trigger icon.
- **Faceted Search Modal:** Live query auto-complete with instant thumbnail preview cards.
- **Product Listing Page (PLP):** Filter sidebar (Category, Size, Color, Price Slider, Material, Fit, In-Stock), mobile sliding filter drawer, 3–4 column responsive grid with thumbnail hover flip, marketing badges (*BESTSELLER*, *30% OFF*), and Quick View modal.

2.3 Phase 3: Product Detail Page & Cross-Selling Engine (Weeks 5–6)

- **Visual Media Gallery:** Multi-angle thumbnail carousel, hover zoom, 360-degree item video display, and dynamic swatch image updates.
- **Interactive Fit Tool:** Size selection buttons with live inventory status per size/color, accompanied by an interactive *Fit Calculator Modal* (calculates recommended size from height/weight).
- **Social Proof & Cross-Sell:** Review breakdown with verified buyer badges, Q&A accordion, and dynamic *"Wear It With"* / *"Complete The Look"* carousels.

2.4 Phase 4: Cart Engine, Multi-Step Checkout & Customer Portal (Weeks 7–8)

- **Slide-out Mini-Cart Drawer:** Animated side drawer with quantity modifiers, item removal, and real-time *Free Shipping Progress Meter*.
- **4-Step Checkout Flow:**
 1. *Shipping Information:* Guest checkout toggle and Google Maps Places API auto-completing address lookup.
 2. *Delivery Method:* Standard (\$5.00), Express (\$15.00), and Local Pickup (Free).
 3. *Payment Integration:* Credit card fields, Apple Pay / Google Pay, and Klarna/Afterpay BNPL tabs.
 4. *Order Review:* Promo code validation, tax/shipping calculation, receipt confirmation modal, and automated Gmail confirmation email dispatch.
- **Customer Account Portal & Live Tracking:** Order history with embedded Google Maps package route tracker, live status checkpoints, self-service return wizard, saved addresses, and wishlist.
- **SMS & Email Dispatch Notifications:** Automatic SMS alerts via Twilio/SMS gateway triggered on order state transitions (Confirmed → Dispatched → Out for Delivery → Delivered).

2.5 Phase 5: Administrative Dashboard & Back-Office Ops (Weeks 9–10)

- **Header Portal Switcher:** Toggle button to switch between Customer Portal and Admin Dashboard.
- **Catalog Matrix Builder:** Multi-variant product builder for *Size* × *Color* × *Material* × *Fit*.
- **Inventory & Order Management:** Low-stock alerts (< 10 units), order state machine (Pending → Processing → Shipped → Delivered), and thermal shipping label printable views.
- **Marketing & Analytics Suite:** Rule-based promo code builder, abandoned cart sequence manager, and Recharts executive graphs (Revenue, AOV, Conversion Rate).

2.6 Phase 6: Quality Assurance, Optimization & Deployment (Weeks 11–12)

- Core Web Vitals audit (LCP < 2.0s, FID < 100ms, CLS < 0.1).
- Accessibility check for WCAG 2.1 AA compliance (keyboard navigation, screen reader labels).
- Production deployment on Vercel or AWS with CI/CD pipeline setup.

3 Database Schema Blueprint (PostgreSQL / Prisma ORM)

```
// Prisma Schema Excerpt
model Product {
  id          String          @id @default(uuid())
  title       String
  slug        String          @unique
  description String
  categoryId  String
  basePrice   Decimal         @db.Decimal(10, 2)
  isFeatured  Boolean         @default(false)
  badge       String?         // e.g., "BESTSELLER"
  variants    ProductVariant[]
  reviews     Review[]
  createdAt   DateTime        @default(now())
}

model ProductVariant {
  id          String          @id @default(uuid())
  productId   String
  sku         String          @unique
  size        String         // XS, S, M, L, XL
  color       String
  material    String?
  fitType     String?
  price       Decimal         @db.Decimal(10, 2)
  stockQuantity Int          @default(0)
  images      String[]
}

model Order {
  id          String          @id @default(uuid())
  orderNumber String          @unique
  status      OrderStatus    @default(PENDING)
  totalAmount Decimal         @db.Decimal(10, 2)
```

```
items      OrderItem[]
createdAt  DateTime      @default(now())
}
```

4 Primary API Endpoint Specifications

Method	Endpoint	Purpose & Parameters
GET	/api/products	Fetch paginated catalog with query filters (category, size, color, minPrice, maxPrice, sort).
GET	/api/products/:slug	Fetch complete product details, matrix variants, and cross-sell items.
POST	/api/cart/validate	Validate item inventory levels and compute discounted subtotal.
POST	/api/checkout/create-intent	Initialize payment gateway session and lock temporary stock reservation.
POST	/api/size-recommender	Calculate recommended size given height, weight, and fit preference.
POST	/api/notifications/email	Trigger automated transactional email (Gmail API / SendGrid) with PDF invoice.
POST	/api/notifications/sms	Send real-time SMS status update to customer phone number via SMS gateway.
GET	/api/tracking/:orderId/map	Fetch live Google Maps geolocation coordinates and delivery route status.
GET	/api/admin/analytics	Fetch aggregated KPIs (Revenue, AOV, Conversion, Stock Velocity).
PUT	/api/admin/orders/:id/status	Update order lifecycle state (Pending → Processing → Shipped) and trigger notification webhooks.

Table 2: Core API Endpoints

5 Definition of Done (DoD) Checklist for Developers

Before marking any sprint feature as complete, the development team must ensure:

- 1. **Functional Accuracy:** Feature matches all requirements in the specification.
- 2. **Responsive Integrity:** Tested on Mobile viewports (375px–430px), Tablet (768px–1024px), and Desktop (1440px+) without layout breaks.
- 3. **State Persistence:** Shopping cart, wishlist, active currency, and customer session persist across browser reloads via local storage sync.
- 4. **Console Cleanliness:** Zero console warnings, no unhandled promise rejections, and proper unmount cleanups for event listeners.
- 5. **Type Safety:** TypeScript interfaces or JSDoc types fully defined for all API responses and component props.
- 6. **Performance Standard:** Dynamic image loading with auto WebP compression and responsive image sources.